

Roteiro de Apresentação — Nota Máxima

Projeto 4 — Cálculo de Folha de Pagamento em C, Java, Haskell e Prolog

Paradigmas

Correção

Comparação crítica

Didática

Objetivo do roteiro

Este roteiro foi pensado para uma apresentação de alto desempenho: explicar o problema, provar que as quatro soluções são equivalentes, comparar os paradigmas com critérios claros e demonstrar domínio técnico sem se perder em detalhes de sintaxe.

Estratégia para nota máxima

1. **Comece pelo problema, não pela linguagem:** mostre entradas, saídas e fórmulas antes dos códigos.
2. **Use o mesmo caso de teste como fio condutor:** Ana, Bruno e Carla validam todas as versões.
3. **Compare arquitetura, estado e manutenção:** isso mostra compreensão de paradigmas, não apenas implementação.
4. **Explique pontos fortes e fricções:** reconhecer limitações demonstra maturidade técnica.
5. **Finalize com critério de escolha:** funcional é natural para transformação de listas; OO é boa para evolução de regras; C dá controle; Prolog explicita relações.

Roteiro slide a slide

Slide	Tempo	Fala sugerida e objetivo
1. Abertura — Projeto 4	30s	Apresentar tema, disciplina e integrantes. Começar dizendo que o objetivo não foi apenas fazer uma folha de pagamento, mas implementar a mesma regra em quatro paradigmas para comparar formas de pensar.
2. O problema	35s	Explicar que a tarefa é calcular salário líquido a partir de salário bruto, dependentes e horas extras. Enfatizar que manter as regras iguais é o que torna a comparação justa.
3. Objetivo do sistema	45s	Mostrar entradas e saídas. Destacar que cada implementação precisa produzir extras, INSS, IRRF, líquido, total da folha, maior e menor salário líquido.
4. Regras principais	60s	Explicar as fórmulas: valor da hora, adicional de 50% nas horas extras, base de IRRF com dedução por dependente e líquido final. Dizer que INSS e IRRF usam faixas simplificadas.
5. Caso de teste obrigatório	45s	Apresentar Ana, Bruno e Carla. Usar os valores como prova objetiva de equivalência: total 6564.90, maior Ana, menor Carla.

Slide	Tempo	Fala sugerida e objetivo
6. Quatro paradigmas	35s	Introduzir a comparação: C representa o imperativo, Java a orientação a objetos, Haskell o funcional e Prolog o lógico. A mesma regra muda de arquitetura.
7. Visão geral das soluções	50s	Comparar ideia central e LOC. Ressaltar que menos linhas não significa automaticamente maior simplicidade; depende da familiaridade com o paradigma.
8. C — Imperativo	75s	Explicar structs, arrays de faixas, laço for, acumuladores e atualização de maior/menor. Ponto forte: fluxo explícito. Fricção: mutação e responsabilidade manual.
9. Java — Orientado a objetos	75s	Explicar records para dados, interface RegraDesconto, classes RegraINSS/RegraIRRF e CalculadoraFolha. Ponto forte: extensibilidade por estratégia. Fricção: mais estrutura.
10. Haskell — Funcional	75s	Explicar dados imutáveis, funções puras, listas, map para transformar funcionários em resultados e sum/maximumBy/minimumBy para agregações. Ponto forte: problema vira pipeline de dados.
11. Prolog — Lógico	75s	Explicar fatos, predicados, consulta de faixas, corte para escolher a primeira faixa e maplist para aplicar a regra. Ponto forte: regras declarativas. Fricção: ordem e instanciação na aritmética.
12. Comparação das faixas	50s	Mostrar que C/Haskell usam tabelas, Java encapsula em objetos/estratégias e Prolog registra fatos. Destacar impacto de manutenção ao adicionar nova faixa.
13. Estado e processamento	50s	Comparar mutação explícita em C com transformação declarativa em Haskell/Prolog e streams em Java. Explicar que todos seguem funcionário → resultado → agregados.
14. O que cada paradigma evidenciou	45s	Resumir brilho e fricção de cada linguagem: controle em C, extensibilidade em Java, pureza em Haskell e regras declarativas em Prolog.
15. Resultados de execução	40s	Dizer que as quatro execuções foram verificadas e produziram os mesmos resultados. Esse é o principal critério de correção do trabalho.
16. Conclusão	55s	Fechar defendendo que não existe paradigma universalmente melhor: existe adequação ao problema. Para este domínio, funcional ficou natural; para evolução de regras, OO é forte.
17. Perguntas	20s	Encerrar com abertura para perguntas e estar pronto para explicar fórmulas, faixas, equivalência dos resultados e diferenças arquiteturais.

Fala de abertura sugerida

“Boa apresentação. Nosso trabalho implementa uma folha de pagamento em quatro linguagens: C, Java, Haskell e Prolog. O objetivo principal não é só chegar ao salário líquido correto, mas observar como cada paradigma organiza dados, regras, fluxo de cálculo e agregações. Por isso, usamos as mesmas fórmulas e o mesmo caso de teste em todas as versões.”

Pontos que precisam ser obrigatoriamente abordados

- Entradas: nome, salário bruto, dependentes e horas extras.
- Saídas: extras, INSS, IRRF, líquido, total da folha, maior e menor líquido.
- Fórmulas de cálculo e dedução por dependente de 189,59.
- Uso das mesmas faixas simplificadas de INSS e IRRF em todas as implementações.
- Equivalência dos resultados: total 6564.90; maior salário líquido Ana; menor Carla.
- Diferença entre fluxo imperativo, encapsulamento orientado a objetos, composição funcional e relações lógicas.

Perguntas prováveis da banca e respostas curtas

Pergunta	Resposta sugerida
Por que usar quatro paradigmas?	Para mostrar que a mesma regra pode ser resolvida com modelos mentais diferentes: comandos, objetos, funções e relações.
Qual solução ficou mais adequada?	Para este domínio, Haskell ficou natural porque a folha é uma transformação de lista. Para evolução de regras, Java é forte pelo encapsulamento.
Como garantiram equivalência?	As tabelas de faixas, fórmulas e dados de entrada são iguais, e as quatro execuções retornam os mesmos totais e extremos.
Por que Prolog exige cuidado?	Porque cálculos aritméticos em Prolog dependem de variáveis já instanciadas e da ordem dos predicados.
Qual o risco em C?	Controle manual de arrays, índices e estado; é claro e eficiente, mas exige disciplina.

Fechamento sugerido

“A principal conclusão é que paradigma não é apenas sintaxe. Ele muda como representamos o domínio, onde colocamos regras, como processamos listas e como justificamos manutenção futura. A solução correta é a que melhor combina com o tipo de problema e com as mudanças esperadas.”

Checklist final antes de apresentar

- Não ler os slides: usar os slides como apoio visual.
- Repetir a ideia de equivalência dos resultados.
- Não gastar tempo excessivo em sintaxe; explicar intenção e arquitetura.
- Mostrar um ponto forte e uma fricção de cada paradigma.
- Encerrar com conclusão comparativa, não apenas “funcionou”.